

OpenMPI User Guide @ IST RNL

Parallel and Distributed Computing (PDC)

2024/2025

This document provides a jump-start for running MPI applications at RNL. The installed version of MPI is OpenMPI 5.0.6. There are two ways in which you can run MPI applications at RNL¹ :

- using the regular MPI commands
- a cluster over all machines in all RNL labs, managed by Slurm (a total of 90 machines, 438 cores).

1 Running MPI Manually on the Lab Machines

1.1 Compiling

The compilation command for a program in C is `mpicc`. `mpicc` accepts all `gcc` flags. For example:

```
$ mpicc -g -o hello_world hello_world.c
```

compiles file `hello_world.c` with debugging information, and creates an executable named `hello_world`.

You can compile your program at any of the lab machines, or in the cluster machine.

1.2 Environment

MPI was designed for parallel execution across different machines, however MPI expects the executable to be in the same directory in all machines. For this reason, all the files related to an MPI execution need to reside below the “cirrus” file system, accessed through the shell variable `$CLUSTER_HOME`. Hence, after logging in at RNL, you need to:

```
$ cd $CLUSTER_HOME
```

Additionally, remote execution requires access by `ssh` without password, and the AFS filesystem makes this trickier. You need to execute (once!) the following steps:

¹<https://rnl.tecnico.ulisboa.pt/servicos/cluster>.

```

$ cd $CLUSTER_HOME
$ mkdir .ssh
$ chmod og-rwx .ssh
$ cd .ssh
$ test -f id_ed25519 -a -f id_ed25519.pub || ssh-keygen -t ed25519 -f ./id_ed25519 -N ""
$ cat id_ed25519.pub >> authorized_keys
$ chmod og-rwx authorized_keys

```

You will still get some “permission denied” errors related to `.bashrc`, it is safe to ignore these.

1.3 Executing an MPI Application

An MPI application is launched using the command `mpirun`. The switch `-n <p>` can be used to specify the number of processes to be created, which, by default, are all created on the local machine. For example,

```
$ mpirun -n 4 hello_world
```

will create four processes on the local machine, all running the same executable `hello_world`.

If the application requires arguments, they should come after the program’s name.

`mpirun` allows the switch `-host`, which must be followed by a list of machines separated by commas where to execute your program. You can use the machines in all RNL labs. These are named `labYpZZ`, where: `Y` is the lab number, 1 through 5 (no 2); `ZZ` is the PC number, 1 through 10 (some labs have more). The command:

```
$ mpirun -host lab1p8,lab1p10 hello_world
```

will create two processes, one on machine `lab1p8` and another on `lab1p10`, both running the program `hello_world`.

It is also possible to create a file with the list of machines, one per line, where the application should run. For example, if the contents of file `my-nodes.txt` is:

```
lab1p8 slots=1
lab1p10 slots=1
```

then the command:

```
$ mpirun -hostfile my-nodes.txt hello_world
```

will have the same behavior as described above.

2 Running the Application using Slurm

The machines over all the RNL's labs are set up to work as a cluster. Again, you can only submit jobs from the `cluster` machine your MPI programs need to be executed/submitted from a directory under the "cirrus" filesystem, as explained above.

To prevent jobs from interfering with one another, the jobs in the cluster are managed by the Slurm system². To run a job, use:

```
$ srun -n <number of nodes> <executable> <args>
```

This is an interactive command and you may have to wait a while before enough resources are available (all Slurm jobs are submitted to the same queue).

Alternatively, you can submit a batch job, and your program will be executed in the background when its turn arrives. For this you need to provide a simple script, for instance:

```
#!/usr/bin/env bash
#SBATCH --job-name=<myJob>
#SBATCH --output=mpi_%j.out
#SBATCH --error=mpi_%j.err
#SBATCH --ntasks=20
srun <executable name>
```

where:

- `job-name` is just for your reference.
- `output` and `error` specifies the files where the `stdout` and `stderr` of your program are redirected to, respectively (%j is replaced by the job number).
- `ntasks` is the number of MPI tasks you require.

You then submit the script using:

```
$ sbatch <script>
Submitted batch job xx
```

You can check your place in the Slurm queue using:

```
$ squeue
```

To cancel a previously submitted job use:

```
$ scancel <job number>
```

²<https://slurm.schedmd.com/documentation.html>

The job number is given by the `sbatch` command (xx above), but you can also check this number with the `squeue` command.

When combining MPI with OpenMP, another important parameter for the batch file is:

```
#SBATCH --cpus-per-task=<n>
```

where `n` defines the default number of threads per process (the default is one). For interactive execution, command `srun` allows the option `-cpus-per-task=<n>`.